

CineMax

Progetto di laboratorio A
sede di Varese
manuale tecnico

Candidati:

Francesco Ferrari

Matricola 721086

Nithay Patricia Gomez Silva

Matricola 768081

Pasquale Di Tuccio

Matricola 766922

Luca

Matricola 765700

Indice

1	Installazione	3
1.1	Requisiti di sistema	3
1.2	Setup ambiente	3
1.3	Installazione programma	3
2	Esecuzione ed uso	4
2.1	Setup e lancio del programma	4
2.2	Classi e funzionalità	5
2.2.1	Le classi	5
2.2.2	Il ciclo di esecuzione principale	9
2.2.3	Funzionalità dell'utente di tipo cliente	20
2.2.4	Funzionalità dell'utente di tipo proiezionista	23
2.2.5	Funzionalità dell'utente di tipo bigliettaio	26
2.3	Data set di test	28
3	Limiti della soluzione proposta	29
4	Bibliografia	30

Capitolo 1

Installazione

In questo capitolo sono presentati i requisiti necessari all'installazione dell'applicazione e la modalità di installazione sui PC.

1.1 Requisiti di sistema

Per eseguire il programma è necessario un computer con sistema Microsoft Windows che abbia installata la Java Virtual Machine.

1.2 Setup ambiente

Le cartelle bin (contiene l'eseguibile), src(contiene il codice sorgente) e data(contiene i file csv che simulano i database utenti, palinsesto e prenotazioni) devono sempre trovarsi all'interno della stessa cartella, altrimenti l'eseguibile non sarà eseguito.

1.3 Installazione programma

Per installare il programma è sufficiente scaricare il repository da GitHub ed eventualmente copiare le cartelle data,src e bin nella cartella di lavoro.

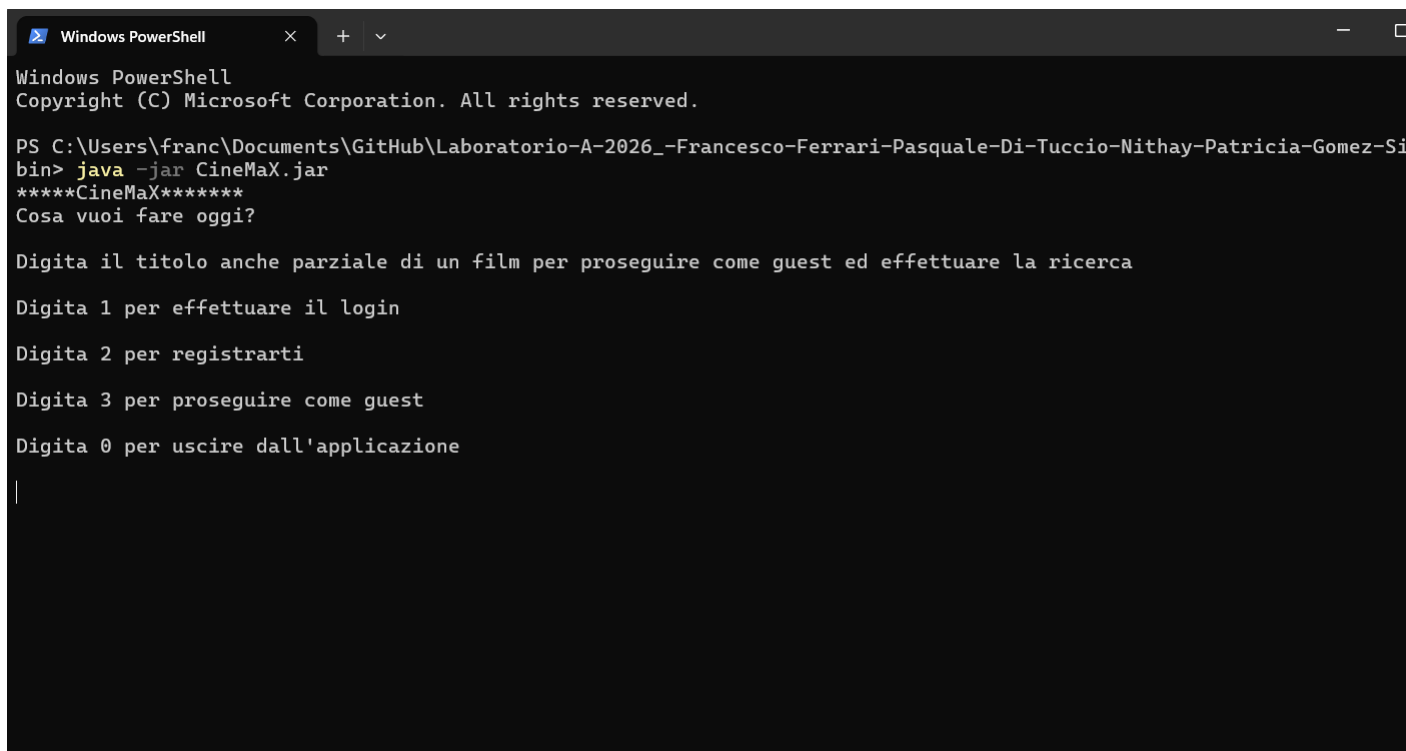
Capitolo 2

Esecuzione ed uso

In questo capitolo sono esposti i principali metodi che garantiscono il funzionamento dell'applicazione. Viene anche discusso come è stato creato l'eseguibile e le modalità di setup e installazione dell'applicazione.

2.1 Setup e lancio del programma

Per lanciare il programma è necessario aprire la shell nella cartella bin e digitare il comando `java -jar CineMaX.jar`



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\franc\Documents\GitHub\Laboratorio-A-2026_-Francesco-Ferrari-Pasquale-Di-Tuccio-Nithay-Patricia-Gomez-Si
bin> java -jar CineMaX.jar
*****CineMaX*****
Cosa vuoi fare oggi?

Digita il titolo anche parziale di un film per proseguire come guest ed effettuare la ricerca

Digita 1 per effettuare il login

Digita 2 per registrarti

Digita 3 per proseguire come guest

Digita 0 per uscire dall'applicazione
|
```

Figura 2.1: Avvio del codice

2.2 Classi e funzionalità

In questa sezione sono descritte le parti fondamentali dell'applicazione, che ne garantiscono il funzionamento corretto; Viene discussa la struttura e l'ereditarietà delle classi.

2.2.1 Le classi

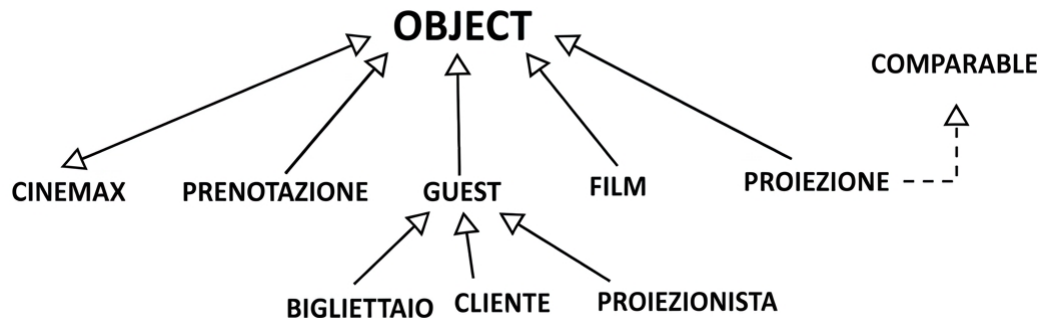


Figura 2.2: Albero dell'ereditarietà del progetto

In questa applicazione JAVA (package CineMaX) la classe Object è estesa direttamente dalle classi Film, Prenotazione, Proiezione (che implementa l'interfaccia Comparable), CineMaX e Guest. Le prime tre servono per creare oggetti di nome corrispondente: gli oggetti di tipo Film servono per costruire oggetti di tipo Proiezione, che rappresentano i film che è possibile inserire o modificare nel palinsesto, mentre il tipo Prenotazione rappresenta le prenotazioni effettuate da un cliente. Infine CineMaX è la classe che contiene il metodo main e i metodi necessari alla registrazione e al login di un account utente (si veda la sezione 2.2.1).

La classe Guest ha tre sottoclassi dirette: Cliente, Proiezionista e Bigliettaio; Guest fornisce i metodi per effettuare la ricerca di una proiezione nel palinsesto (si veda la sezione 2.2.1), Cliente consente di effettuare, modificare o eliminare prenotazioni oltre alla ricerca di proiezioni (si veda la sezione 2.2.1), Proiezionista consente di interagire col palinsesto inserendo, rimuovendo o modificando una proiezione (si veda la sezione 2.2.1) e infine Bigliettaio consente di visualizzare e ricercare prenotazioni effettuate dai clienti (si veda la sezione 2.2.1).

La classe Film

La classe Film estende direttamente Object e serve unicamente per creare e operare su oggetti utilizzati nella costruzione di oggetti di tipo Proiezione, gli oggetti di tipo Film modellano il film da inserire nel palinsesto delle proiezioni e contengono le informazioni generali di un film, che sono:

- **private String Titolo**
- **private String Genere**
- **private String Regista**
- **private String Anno di pubblicazione**
- **private String Durata**
- **private int Età**

La classe contiene inoltre un costruttore, i metodi get e set per i campi e un metodo per stampare i campi mostrato in figura 2.4.

```

1 package CineMaX;
2 /**
3  * Questa classe costruisce oggetti di tipo Film
4  * che sono utilizzati per costruire oggetti di tipo
5  * Proiezione
6  */
7 public class Film { // Questa classe costruisce oggetti di tipo Film
8     //-----
9     // Campi
10    private String Titolo;
11    private String Genere;
12    private String Regista;
13    private int Anno;
14    private int Durata; // In minuti
15    private int Età; // Età minima
16    //-----
17    // Costruttori
18 /**
19  * Questo costruttore forma oggetti di tipo Film
20  *
21  * @param titolo il titolo del film come String
22  * @param genere il genere del film come String
23  * @param regista il regista del film come String
24  * @param anno l'anno in cui è uscito il film come int
25  * @param durata la durata in minuti del film come int
26  * @param età l'età minima consigliata per la visione del film come int
27  */
28 public Film(String titolo, String genere, String regista, int anno, int durata, int età) {
29     Titolo=titolo;
30     Genere=genere;
31     Regista=regista;
32     Anno=anno;
33     Durata=durata;
34     Età=età;
35 }
36
37 //-----
38 // Metodi
39
40 // Getter
41
42 /**

```

Figura 2.3: I campi e il costruttore della classe film

```

127 /**
128  * Questo metodo stampa a schermo le caratteristiche di un Film
129  */
130 public void visualizzaFilm() {
131     System.out.println("Film: " + Titolo);
132     System.out.println("Genere: " + Genere);
133     System.out.println("Regista: " + Regista);
134     System.out.println("Anno: " + Anno);
135     System.out.println("Durata (min): " + Durata);
136     System.out.println("Età minima: " + Età);
137
138 }

```

Figura 2.4: Metodo che stampa le caratteristiche di un film

La classe Proiezione

La classe proiezione crea gli oggetti che modellano le proiezioni del palinsesto, questa classe estende direttamente la classe Object e implementa l'interfaccia Comparable<Proiezione> per consentire l'ordinamento cronologico automatico degli spettacoli in memoria. I suoi campi sono:

- **private Film film**
- **private LocalDateTime dataOra**
- **private double PrezzoBiglietto**
- **private int numeroPosti**
- **public static final int CAPIENZA_MASSIMA=200**
- **private static final String FILE_PROIEZIONI**

che rappresentano rispettivamente il film proiettato, la data e l'orario della proiezione in formato AAAA-MM-GG HH:MM:SS il prezzo del biglietto, il numero di posti prenotati, la capienza massima della sala del cinema e il percorso del file palinsesto proiezioni.csv. Questa classe, oltre ai metodi get, set e al costruttore, contiene i metodi accessori della classe Proiezionista.

```

19 public class Proiezione implements Comparable<Proiezione> {
20
21     private Film film;
22     private LocalDateTime dataOra;
23     private double prezzoBiglietto;
24     private int numeroPosti;
25
26     // Capienza massima del cinema monosala
27     public static final int CAPIENZA_MASSIMA = 200;
28     private static final String FILE_PROIEZIONI = "..\\data\\Proiezioni.csv";
29
30     /**
31      * Costruttore della classe Proiezione.
32      *
33      * @param film Il film proiettato.
34      * @param dataOra Data e ora dello spettacolo.
35      * @param prezzoBiglietto Prezzo del singolo biglietto.
36      * @param numeroPosti Numero totale di posti in sala per la proiezione.
37      */
38     public Proiezione(Film film, LocalDateTime dataOra, double prezzoBiglietto, int numeroPosti) {
39         this.film = film;
40         this.dataOra = dataOra;
41         this.prezzoBiglietto = prezzoBiglietto;
42         this.numeroPosti = numeroPosti;
43     }
44

```

Figura 2.5: I campi e il costruttore della classe Proiezione

La classe Prenotazione

La classe Prenotazione estende direttamente Object costruisce oggetti di tipo prenotazione usati nel menù utente della classe cliente, descritta in sezione 2.2.1, contiene anche i metodi usati da quest'ultima per effettuare verifiche sulle stringhe inserite dall'utente e per interagire con il file prenotazioni.csv; questa classe fornisce inoltre i metodi per inserire, modificare o cancellare una prenotazione attraverso il menù utente; i campi di prenotazione sono:

- **private String IDutente**
- **private String Nome**
- **private String Cognome**
- **private LocalDateTime Proiezione_ Data**
- **private String Proiezione_ Titolo**
- **private int ID_ prenotazione**
- **private int numeroPosti**
- **private double Prezzo_ Biglietto**

che modellano rispettivamente l'ID univoco assegnato all'utente in fase di registrazione (si veda la sezione 2.2.2), il nome e il cognome dell'utente, la data e l'orario della proiezione in formato AAAA-MM-GG HH:MM:SS, l'ID della prenotazione, che viene assegnata come intero in ordine crescente dal metodo public static int generaNuovoID(), il numero di posti prenotati e il prezzo del biglietto. Questa classe contiene i metodi get e set per i suoi campi.

La classe Guest

La classe guest è una classe senza campi ne costruttori che contiene i metodi per effettuare la ricerca di una proiezione. I metodi di questa classe sono utilizzati nell'interfaccia guest nel metodo main (sezione 2.2.2 e dalla classe Cliente, come discusso in sezione 2.2.3).

La classe Cliente

La classe Cliente estende direttamente la classe Guest ed eredita da essa i metodi di ricerca ed esplorazione delle proiezioni. Modella l'utente registrato come cliente del cinema.

I suoi campi privati sono:

- **private String Nome**
- **private String Cognome**
- **private String IDutente**

che modellano rispettivamente il nome dell'utente e l'ID univoco assegnato al cliente quando si registra dal metodo statico Registrati() della classe CineMaX . La classe incapsula la logica relativa all'inserimento, alla rimozione e alle modifiche delle prenotazioni.

La classe Proiezionista

```
public class Proiezionista extends Guest {

    private String nome;
    private String idProiezionista;

    /**
     * Costruttore vuoto della classe Proiezionista.
     */
    public Proiezionista() {}

    /**
     * Costruttore con parametri per la classe Proiezionista.
     *
     * @param nome Il nome del proiezionista.
     * @param idProiezionista L'identificatore univoco del proiezionista.
     */
    public Proiezionista(String nome, String idProiezionista) {
        this.nome = nome;
        this.idProiezionista = idProiezionista;
    }
}
```

Figura 2.6: I campi e il costruttore della classe Proiezionista

La classe Proiezionista estende direttamente la classe Guest ed eredita da essa i metodi di ricerca ed esplorazione delle proiezioni. Rappresenta l'operatore del cinema incaricato della gestione tecnica del palinsesto.

I suoi campi privati sono:

- **private String Nome**
- **private String idProiezionista**

che modellano rispettivamente il nome dell'utente e l'ID univoco assegnato al proiezionista nel formato **PRO[numero]. La classe incapsula la logica relativa alle modifiche del palinsesto.

La classe Bigliettaio

La classe CineMaX

La classe CineMaX è la classe in cui si trova il metodo Main (che contiene il ciclo di esecuzione principale) e tutti i metodi necessari al login e alla registrazione di un account utente; tutti i metodi di questa classe a parte Main hanno visibilità private in quanto l'unica classe in cui devono essere utilizzati è questa (vengono usati all'interno del Main). Maggiori dettagli riguardo questa classe saranno forniti nella sezione 2.2.2 in cui si illustrano le funzionalità del ciclo di esecuzione principale dell'applicazione.

2.2.2 Il ciclo di esecuzione principale

Il metodo main contiene il ciclo di esecuzione principale dell'applicazione, che garantisce l'esecuzione fino a quando la variabile boolean On, inizializzata a true, non viene posta false nel logout. Prima di entrare nell'esecuzione il programma dichiara anche la variabile di tipo Guest loggeduser che sarà usata per il login dell'utente.

Durante l'esecuzione dell'applicazione viene mostrato all'utente il menù principale mostrato in figura 2.7, che consente di navigare nel programma inserendo da tastiera il valore corrispondente alla funzionalità scelta dall'utente; ogni scelta possibile corrisponde al case di uno switch/case e ogni sottomenù è allo stesso modo uno switch/case che rimane in esecuzione fino a quando l'utente decide di ritornare al menù principale.

```
****CineMaX****
Cosa vuoi fare oggi?

Digita il titolo anche parziale di un film per proseguire come guest ed effettuare la ricerca

Digita 1 per effettuare il login

Digita 2 per registrarti

Digita 3 per proseguire come guest

Digita 0 per uscire dall'applicazione

|
```

Figura 2.7: Il menù principale

Come si può vedere in figura 2.8 il menù principale è costruito con System.out.println(String) e l'input da tastiera per mezzo dello Scanner(System.in) Kinput, il cui valore entra nello switch principale che lo confronta con una stringa.

```

952 public static void main(String[] args) {
953     boolean On=true; //questa variabile serve per effettuare l'interruzione dell'esecuzione dell'applicazione
954     Guest loggeduser; //questa variabile salva l'utente correntemente loggato
955     while(On==true) {
956         System.out.println("*****CineMaX*****");
957         //cosa vuoi fare? loggarti, registrarti o proseguire come guest?
958         //rimango nel ciclo di esecuzione principale fino a quando non chiudo l'app
959         Scanner Kinput=new Scanner(System.in); //refresh dello scanner ad ogni iterazione per pulire la storia delle operazioni
960         System.out.println("Cosa vuoi fare oggi?\n ");
961         System.out.println("Digita il titolo anche parziale di un film per proseguire come guest ed effettuare la ricerca\n");
962         System.out.println("Digita 1 per effettuare il login\n ");
963         System.out.println("Digita 2 per registrarti\n");
964         System.out.println("Digita 3 per proseguire come guest\n ");
965         System.out.println("Digita 0 per uscire dall'applicazione\n ");
966         String swtch=Kinput.nextLine(); //Questa variabile serve come selettore per la modalit  in cui si intende usare l'app,   di t
967         switch(swtch) {
968             case "1": // login

```

Figura 2.8: Il men  principale nel metodo main

```

769 private static void Registrati() {
770     String dataparsed;
771     Scanner obj=new Scanner(System.in); //creo un oggetto della classe Scanner, che serve tra le altre
772     LocalDate dataoggi=LocalDate.now(); //trovo la data attuale
773     String nome;
774     while(true) { // il nome deve avere almeno 2 lettere
775         System.out.println("Nome: ");
776         nome=obj.nextLine(); // il metodo nextLine() restituisce la stringa corrispondente all'ultimo i
777         if(nome.matches("[0-9\\p{Punct}\\s].*")) { //se il nome contiene numeri o simboli speciali o
778             System.out.println("Il nome non pu  contenere numeri, caratteri speciali o spazi, riprova!
779             continue;
780         }
781         if(nome.length()<2) {
782             System.out.println("Il nome fornito deve avere almeno due caratteri, riprova!");
783             continue;
784         }
785         break;
786     }
787     String cognome;
788     while(true) {
789         System.out.println("Cognome: ");
790         cognome=obj.nextLine();
791         if(cognome.matches("[0-9\\p{Punct}\\s].*")) { //se il cognome contiene numeri o simboli speci
792             System.out.println("Il cognome non pu  contenere numeri, caratteri speciali o spazi, riprova!
793             continue;
794         }
795         if(cognome.length()<2) {
796             System.out.println("Il cognome fornito deve avere almeno due caratteri, riprova!");
797             continue;
798         }
799         break;
800     }
801     String data;
802     while(true) {
803         System.out.println("Data di nascita (AAAA-MM-GG): ");
804         data=obj.nextLine();
805         dataparsed= CineMaX.controlloData(data);
806         if(dataparsed==null) continue; // se il controllo della data fallisce riprova
807         LocalDate dataID=LocalDate.parse(dataparsed); //converto la stringa in formato localdate
808         Period et =Period.between(dataID, dataoggi); // trovo anni mesi e giorni nati dalla data in
809

```

Figura 2.9: Inizio del metodo Registrati

Registrarsi

La registrazione di un nuovo utente, corrispondente alla scelta 2 nel menù principale, corrisponde alla scrittura delle informazioni dell'utente (nome, cognome..) nelle colonne del file Utenti.csv e viene effettuata invocando il metodo private static void registarti() della classe CineMaX. Come si può vedere nell'immagine ??, questo metodo usa lo scanner(System.in) obj per salvare l'input da tastiera dell'utente; il codice chiede all'utente di inserire le sue informazioni una alla volta, ogni inserimento è all'interno di un ciclo while(true) così che non è possibile passare all'informazione successiva fino a quando la precedente non viene inserita correttamente. Questo è necessario soprattutto per evitare che l'utente inserisca stringhe vuote nel file Utenti.csv, che potrebbero pregiudicarne la lettura causando il crash dell'applicazione.

```

802 String data;
803 while(true) {
804     System.out.println("Data di nascita (AAAA-MM-GG): ");
805     data=obj.nextLine();
806     dataparsed= CineMaX.controllaLoData(data);
807     if(dataparsed==null) continue; // se il controllo della data fallisce riprova
808     LocalDate dataLD=LocalDate.parse(dataparsed); // converto la stringa in formato localdate
809     Period età=Period.between(dataLD, dataoggi); // trovo anni mesi e giorni passati dalla data ins
810     //se hai più di 125 anni ripeti la registrazione
811     if(età.getYears()>=125) {
812         System.out.println("Attenzione, risulta che hai più di 125 anni, riprova!");
813         continue;
814     }
815     //se hai meno di 18 anni devi loggare con l'account di un maggiorenne per conferma
816     if(età.getYears()<18) {
817         System.out.println("Attenzione, risulta che hai meno di 18 anni anni");
818         System.out.println("per proseguire effettua il login con l'account");
819         System.out.println("di un utente maggiorenne\n");
820         Guest garante=CineMaX.Login();
821         if(garante instanceof Proiezionista || garante instanceof Bigliettaio) {
822             //i dipendenti sono per forza maggiorenni
823             System.out.println("Verifica dell'età completate correttamente!");
824             break;
825         }
826         if(garante instanceof Cliente) { // se l'utente è cliente controllo l'età
827             String keyage=CineMaX.dataDinascita(((Cliente) garante).getIDUtente()); // ottengo la d
828             LocalDate keyageLD=LocalDate.parse(keyage); // converto l'età in formato localdate
829             età=Period.between(keyageLD, dataoggi); // trovo anni mesi e giorni passati dalla data
830             //se hai più di 125 anni ripeti la registrazione
831             if(età.getYears()>=18) { // se l'utente è maggiorenne sono passati 18 anni
832                 System.out.println("Verifica dell'età completate correttamente!");
833                 break;
834             }
835             else {
836                 System.out.println("Attenzione l'utente selezionato ha meno di 18 anni, riprova!");
837                 continue;
838             }
839         }
840     }
841     break;

```

Figura 2.10: Controllo dell'età

CineMaX esegue un controllo dell'età e non consente la registrazione a utenti che hanno più di 125 anni di età per evitare l'inserimento di date inverosimili, come ad esempio 1693-07-16, o la registrazione di minorenni senza il consenso di un maggiorenne; la correttezza del formato della data inserita viene controllata con il metodo statico CineMaX.controllaLoData(String data), quest'ultimo è un parser che converte qualsiasi data inserita dall'utente nel formato corretto (AAAA-MM-GG) e la restituisce come String se

- la data inserita non è formata da soli numeri
- la data inserita ha un numero di caratteri conforme a una data (8<caratteri<10)
- la data inserita non contiene più di due caratteri non numerici (i separatori)
- i separatori non sono due simboli diversi tra loro
- i caratteri non numerici presenti sono nelle posizioni corrette per un separatore

Sebbene il formato di data corretto sia suggerito all'utente, questo parsing si rende necessario in quanto per il controllo dell'età si usa il metodo statico Period.between(LocalDate, LocalDate) della classe Period, che restituisce la differenza in anni-mesi-giorni di due date in formato LocalDate, che in questo caso sono la data di nascita parsata inserita nella variabile dataLD e la data odierna estratta dal sistema all'invocazione del metodo tramite il metodo statico LocalDate.now() e salvata nella variabile LocalDate dataoggi. Qualora la differenza tra le date

risulti maggiore o uguale a 125 anni viene richiesto all'utente di riprovare fino all'inserimento di una data valida, mentre se l'utente risulta minorenne al momento della registrazione viene richiesto il login (si veda la sezione 2.2.2) di un utente maggiorenne: se l'utente è di tipo proiezionista o bigliettaio il controllo è riuscito poichè i dipendenti sono necessariamente maggiorenni, mentre se l'utente è di tipo cliente il programma calcola la differenza delle età come differenza tra date e consente di proseguire se e solo se questa è maggiore o uguale a 18 anni. Il metodo controlla inoltre che lo username non sia già in uso con il metodo statico `CineMaX.usernameinuse(String username)`, che accede al file `Utenti.csv`, controlla se lo username digitato è già presente e restituisce `true` se la risposta è affermativa, `false` altrimenti.

```

95 private static boolean usernameInUse(String username) {
96     try {
97         FileReader filerd = new FileReader(percorsofile);
98         BufferedReader buff= new BufferedReader(filerd);
99         String riga;
100         String[] campiriga=new String[8];
101         buff.readLine();//salto la prima riga del testo
102         while ((riga=buff.readLine())!=null) {
103             campiriga=riga.split(",");
104             if((username.trim()).equals(campiriga[3].trim())) {
105                 buff.close();
106                 filerd.close();
107                 return true;
108             }
109         }
110         buff.close();
111     } catch (FileNotFoundException e) {
112         System.out.println("Errore critico, file utenti.csv non trovato!");
113     } catch (IOException e) {
114         System.out.println("Errore critico, file utenti.csv non valido!");
115     }
116     return false;
117 }

```

Figura 2.11: Controllo username già in uso

Il metodo non permette il salvataggio in chiaro delle password nel file criptandole con l'algoritmo SHA 256 prima della scrittura.

```

167 private static String sha256Hash(String testo) {
168     try {
169         MessageDigest digest = MessageDigest.getInstance("SHA-256");//crea un'istanza dell'algoritmo sha256 utilizzato per criptare (dichiara)
170         byte[] hashed=digest.digest(testo.getBytes(StandardCharsets.UTF_8));//converte in hash il testo da criptare e lo restituisce come array di byte
171         StringBuilder hexString=new StringBuilder();//converte l'array di byte in stringa esadecimale
172         //StringBuilder è un modo più efficiente di lavorare con le stringhe:
173         //String deve allocare memoria per ogni modifica, mentre StringBuilder
174         //modifica direttamente la stringa in memoria
175         for(byte b : hashed) {
176             String s=String.format("%02x", b); //converte il singolo bit in due caratteri esadecimali
177             //%%02x= imposto il carattere di riempimento a 0, la stringa deve avere almeno 2 caratteri, x formato esadecimale lettere minuscole
178             hexString.append(s);
179         }
180         String encoded=hexString.toString(); //converto da StringBuilder a String
181         return encoded;
182     } catch (NoSuchAlgorithmException e) {
183         System.out.println("Errore critico, non è possibile istanziare l'algoritmo SHA256!");
184     }
185     return null;
186 }

```

Figura 2.12: Criptaggio password con SHA 256

Alla fine del processo, il metodo assegna lo username all'utente con il metodo `CineMaX.assegnaIDstr(String nome, String cognome, String username)`, che forma uno username utilizzando le prime due lettere dei suoi input e il numero della riga in cui verrà salvato nel file contato dopo le prime cinque righe riservate ai dipendenti.

Effettuare il login

Il login di un'utente (scelta 1 nel menù principale) è gestito dal metodo `CineMaX.login()`, questo metodo chiede l'inserimento di username e password, cripta la password inserita con SHA 256

```

198 private static String assegnaIDstr(String nome, String cognome, String username) {
199     Path percorso=Paths.get(percorsofile);
200     String ID="";
201     try {
202         long numerorighe=(Files.lines(percorso).count())-7; //conta le righe del file e toglie le righe iniziali per far partire da 1 il
203         String numeroID=Long.toString(numerorighe);
204         return ID=""+nome.charAt(0)+nome.charAt(1)+cognome.charAt(0)+cognome.charAt(1)+username.charAt(0)+username.charAt(1)+numeroID;
205     } catch (IOException e) {
206         System.out.println("Errore critico, file utenti.csv non valido!");
207         e.printStackTrace();
208     }
209     return ID;
210 }

```

Figura 2.13: Assegnazione ID utente

per confrontarla con la password relativa allo username salvata sul file Utenti.csv e se sia lo username e la password coincidono genera un oggetto della classe Guest che istanzia la classe corrispondente al tipo di utente che si sta loggando (Cliente, Bigliettaio o Proiezionista). Il login nella categoria corretta è gestito nel caso 1 dello switch/case principale da una serie di if(loggeduser instanceof classe); la classe Guest non è mai istanziata direttamente dal metodo di login, l'oggetto di tipo Guest viene creato se e solo se si desidera procedere come Guest così da non chiedere mai se un utente è istanza di Guest, che genererebbe ambiguità perchè vero per ogni tipo di utente.

```

900 private static Guest login() {
901     Cliente client;
902     Proiezionista pro;
903     Bigliettaio big;
904     Scanner input=new Scanner(System.in);
905     System.out.println("Username: ");
906     String username=input.nextLine();
907     System.out.println("Password: ");
908     String pword=input.nextLine();
909     //char[] criptpwrd=System.console().readPassword(input.nextLine()); //nasconde la password a console (non funziona su IDE)
910     //String pword=String.valueOf(criptpwrd);
911     pword=CineMaX.sha256Hash(pword); //Cripta la password inserita per confrontarla con quella nel file;
912     try {
913         FileReader filerid = new FileReader(percorsofile);
914         BufferedReader buff= new BufferedReader(filerid);
915         String riga;
916         String[] campiriga=new String[8];
917         buff.readLine(); //salto la prima riga del testo
918         while ((riga=buff.readLine())!=null) {
919             campiriga=riga.split(",");
920             if((username.trim().equals(campiriga[3].trim())&&(pword.trim().equals(campiriga[4].trim())) {
921                 switch(campiriga[7]) {
922                     case "P":
923                         pro=new Proiezionista();
924                         System.out.println("Benvenuto"+campiriga[3]);
925                         return pro;
926                     case "B":
927                         big=new Bigliettaio();
928                         System.out.println("Benvenuto"+campiriga[3]);
929                         return big;
930                     case "C":
931                         client= new Cliente(campiriga[1], campiriga[2], campiriga[0]);
932                         System.out.println("Benvenuto"+campiriga[3]);
933                         return client;
934                 }
935             }
936         }
937         buff.close();
938         filerid.close();
939     } catch (FileNotFoundException e) {
940         System.out.println("Errore critico, file utenti.csv non trovato!");
941     } catch (IOException e) {

```

Figura 2.14: Metodo usato per il login

Login come Guest e ricerca di una proiezione

Se si digita un qualsiasi carattere che non corrisponde alle scelte possibili nel menù principale, si entra nel caso di default dello switch/case principale, che interpreta la stringa data in input dall'utente come il titolo (intero o parziale) di una proiezione ed effettua una ricerca nel palinsesto per mostrare tutte le proiezioni con quel nome disponibili. Per farlo l'utente deve essere loggato come guest quindi per prima cosa si procede a creare un nuovo oggetto di tipo guest e salvarlo nella variabile loggeduser, che sarà usata per invocare il metodo public void cercaProiezione() della classe Guest, che implementa l'interfaccia di ricerca di una proiezione. Dopo aver

```

3
Benvenuto! stai procedendo come Guest

Cosa vuoi fare oggi?

Digita 1 per cercare una proiezione

Digita 2 per registrarti

Digita il titolo anche parziale di un film per effettuare una ricerca rapida

Digita 0 per effettuare il logout

```

Figura 2.15: il menù dell'interfaccia guest

```

1166 default://Se non si fa nessuna scelta si può comunque cercare un film per titolo anche parzia
1167         loggeduser=new Guest();
1168         try {
1169             loggeduser.cercaProiezionePerTitolo(swtch);
1170         } catch (FileNotFoundException e) {
1171             System.out.println("Errore critico, file proiezioni.csv non trovato!");
1172         }
1173         System.out.println("Benvenuto! stai procedendo come Guest\n");
1174         while(true) {
1175             System.out.println("Cosa vuoi fare oggi?\n ");
1176             System.out.println("Digita 1 per cercare una proiezione \n ");
1177             System.out.println("Digita 2 per registrarti\n");
1178             System.out.println("Digita 0 per effettuare il logout\n "); //puoi uscire senza regist
1179             String todo=Kinput.nextLine();
1180             switch(todo){
1181                 case "1":
1182                     try {
1183                         loggeduser.cercaProiezione();
1184                         continue;
1185                     } catch (FileNotFoundException e) {
1186                         System.out.println("Errore critico, file proiezioni.csv non trovato!");
1187                         break;
1188                     }
1189                 case "2":
1190                     CineMaX.Registrati();
1191                     break;
1192                 case "0":
1193                     System.out.println("Arrivederci! Grazie per aver usato la nostra app!");
1194                     break;
1195                 default:
1196                     try {
1197                         loggeduser.cercaProiezione();
1198                         continue;
1199                     }
1200                     catch (FileNotFoundException e){
1201                         System.out.println("Errore critico, file proiezioni.csv non trovato!");
1202                         break;
1203                     }
1204             }
1205         }
1206         break;
1207     }

```

Figura 2.16: Caso default e interfaccia guest

effettuato la ricerca si rimarrà quindi loggati come guest, quindi sarà possibile effettuare una nuova ricerca, registrarsi con il metodo statico `CineMax.Registrati()` (si veda la sezione 2.2.2) o tornare al menù principale. Questa modalità è uguale al caso 3 dello switch/case principale, che corrisponde all'accesso come guest per cui è solamente possibile registrarsi o effettuare la ricerca di una proiezione nel palinsesto.

Quando l'utente accede alla funzionalità di ricerca di una proiezione il programma chiama il metodo `cercaProiezione()` della classe `Guest`. **NB Per motivi sia di necessità, sia di leggibilità il codice, il metodo si estende su troppe righe per permettere l'utilizzo di screenshot del codice all'interno del manuale. Pertanto, per questa parte si consiglia di leggere il manuale guardando contemporaneamente anche il codice del programma.**

Il metodo effettua la ricerca di una proiezione, secondo criteri decisi dall'utente, accedendo al file `proiezioni.csv` per leggere le informazioni sulle proiezioni. Il metodo non ha parametri e come valore di ritorno restituisce un array di oggetti di tipo `Proiezione` che conterrà gli oggetti `Proiezione` che rappresentano le proiezioni che sono risultato della ricerca, oppure null nel caso in cui la ricerca non abbia risultati (perché nessuna proiezione rispetta i requisiti dell'utente oppure perché la ricerca è stata annullata). Il metodo inizia con le dichiarazioni di alcune variabili necessarie allo svolgimento del metodo. Dopodiché è presente il ciclo `do-while` (`sceltaOk==false`) in cui è contenuta la ricerca; la variabile boolean `sceltaOk` viene inizializzata a `true` all'inizio di ogni iterazione e viene messa a `false` all'interno del ciclo in quei casi in cui sia necessario ripetere il ciclo (e quindi cioè ritornare al menù della ricerca). Infine, è presente l'`if-else` (`if(numRisRicerca>0)`)-`else` all'interno del quale: nel caso in cui la ricerca abbia dato almeno un risultato, il programma permette all'utente di selezionare una delle proiezioni trovate con la ricerca per visualizzarne i dettagli e ritorna il vettore contenente le proiezioni risultato della ricerca, altrimenti il programma comunica all'utente la mancanza di risultati ed effettua `return null`. Il corpo del ciclo `do-while` (`sceltaOk==false`) è costituito da:

- inizializzazione (o re-inizializzazione con intento di reset, nelle esecuzioni del ciclo successive alla prima) di variabili necessarie per la ricerca
- stampa a schermo del menù della ricerca e prelievo input della scelta dell'utente
- switch `switch(scelta)` che gestisce la ricerca

Il ciclo si rende necessario per ricominciare da capo per due possibili motivi: nel caso l'utente inserisca un'opzione non valida nel menù della ricerca oppure nel caso l'utente svolga una ricerca che arriva ad un numero di risultati superiore al limite consentito (nel qual caso segnala all'utente la problematica, richiede all'utente di inserire dei criteri più restrittivi in una nuova ricerca, interrompe la ricerca attuale e procede a tornare al menù della ricerca). Il menù della ricerca che viene stampato all'utente è mostrato in figura 2.17.

```
Ricerca di una proiezione
Selezionare un criterio per la ricerca:
1=per titolo
2=per genere di film
3=per data
4=per prezzo del biglietto
5=per una combinazione dei precedenti criteri di ricerca
0=per annullare la ricerca
Inserire criterio scelto:
```

Figura 2.17: il menù di ricerca di una proiezione

Dopo l'inserimento dell'input da parte dell'utente il programma entra nello `switch(scelta)` che effettua la ricerca; la variabile `scelta` conterrà la scelta inserita dall'utente. Nel caso in cui l'utente inserisca un input non valido si andrà al case default: il programma segnala l'errore all'utente,

esegue `break` per uscire dallo `switch` e torna all'inizio del ciclo `do-while` per tornare al menù della ricerca. Nel caso in cui l'utente inserisca un input valido ci sono due possibilità: ha inserito l'opzione per annullare la ricerca oppure ha selezionato uno dei criteri disponibili per la ricerca. Nel primo caso si entra nel case dedicato in cui il programma non farà altro che ritornare `null`, terminando il metodo. Nel secondo caso si entra nel case che si occupa della ricerca secondo il criterio scelto dall'utente. Nei vari case si può osservare un modo di procedere ricorrente: prima l'applicazione ottiene dall'utente le informazioni necessarie per effettuare la ricerca (nella maniera appropriata al criterio scelto), dopodiché viene effettuato il ciclo `while while(scFile.hasNext())` in cui il programma effettua la ricerca secondo il criterio scelto dall'utente, usando le informazioni inserite dall'utente (`while(scFile.hasNext())` va interpretato come “while(il file proiezioni.csv ha ancora qualcosa da leggere)” e quindi cioè “while(ci sono proiezioni da controllare)”). In particolare, nel ciclo `while(scFile.hasNext())` il programma:

- ottiene una proiezione dal file `proiezioni.csv` tramite il metodo `estraiProiezione()` della classe `Guest`; il metodo `estraiProiezione()` restituisce un oggetto di tipo `Proiezione` che rappresenta la proiezione letta/estratta dal file `proiezioni.csv`
- verifica se la proiezione letta/estratta rispetta i requisiti dell'utente della scelta dell'utente
- nel caso in cui la proiezione rispetti i requisiti verifica se si è raggiunto il limite massimo di risultati della ricerca. In caso affermativo il programma segnala all'utente il raggiungimento del limite e quindi l'impossibilità di proseguire la ricerca, richiede di inserire dei criteri più restrittivi in una nuova ricerca e interrompe la ricerca corrente (in particolare `sceltaOk` viene messa a `false`, viene eseguita una `break` per uscire dal ciclo `while` portando il programma ad effettuare la `break` del case in cui si trovava; pertanto il programma esce dallo `switch` e quindi arriva al `while(sceltaOk==false)` che farà sì che si torni al menù della ricerca). In caso contrario il programma salva la proiezione controllata in un vettore e visualizza la proiezione all'utente, numerandola e mostrandone le informazioni con il metodo `visualizzaProiezione()` della classe `Proiezione`
- al termine della ricerca informa l'utente del numero di risultati della ricerca

Pertanto, l'unica differenza tra i case risiede in quali informazioni vengono richieste all'utente. Nella ricerca per titolo viene richiesto di inserire il titolo del film (anche parziale). Nella ricerca per genere viene richiesto di inserire il genere del film (anche parziale). Nella ricerca per data prima viene richiesto di selezionare, tramite un menù, una delle seguenti possibilità

- proiezioni prima di una certa data
- proiezioni dopo una certa data
- proiezioni comprese tra due date

e dopo a seconda dell'opzione scelta viene richiesto di inserire la data o le date da usare come criterio di ricerca. Nella ricerca per prezzo del biglietto prima viene richiesto di selezionare, tramite un menù, una delle seguenti possibilità

- proiezioni con prezzo minore di un certo valore
- proiezioni con prezzo maggiore di un certo valore
- proiezione con prezzo compreso tra due valori

e dopo a seconda dell'opzione scelta viene richiesto di inserire il prezzo o i prezzi da usare come criterio di ricerca. Nel case relativo alla ricerca per combinazione di criteri l'applicazione chiede all'utente di indicare quali dei criteri precedentemente menzionati si desidera utilizzare per la ricerca; nel caso in cui l'utente non selezioni almeno un criterio il programma segnalerà tale cosa e procederà a ripetere la selezione di un criterio. Scelto almeno un criterio da parte dell'utente, a seconda di quali criteri vengono selezionati l'applicazione richiede di inserire, per

ogni criterio scelto, le informazioni necessarie per quel criterio in maniera simile a come avviene nel caso in cui si effettuasse una ricerca secondo solo quel criterio (ad esempio, nel caso in cui si selezionassero come criteri di ricerca il genere e il prezzo l'applicazione chiede per il genere di inserire un genere (anche parziale) e per il prezzo di scegliere se cercare proiezioni con prezzo minore di un certo valore, maggiore valore o compreso tra due valori e a seconda dell'opzione scelta di inserire il prezzo/i prezzi da usare come criterio). Nel case relativo alla ricerca per combinazione di criteri, all'interno del `while(scFile.hasNext())` verranno controllati solo i criteri selezionati dall'utente e, nel caso di più criteri selezionati, non appena il programma verifica che uno dei criteri non è rispettato allora non verranno controllati gli altri criteri inutilmente. Conclusa la ricerca il programma esce dal `while(scFile.hasNext())`. Il programma procede con l'`if-else if(numRisRicerca>0)-else` il quale verifica se la ricerca ha avuto almeno un risultato o meno. Nel caso in cui la ricerca avesse risultati allora l'applicazione chiederà all'utente se desidera visualizzare in dettaglio una delle proiezioni trovate tramite la ricerca e, in caso affermativo, richiederà di inserire il numero della proiezione che desidera visualizzare in dettaglio. Inserito il numero di una delle proiezioni il programma permette all'utente di visualizzare nel dettaglio la proiezione scelta tramite il metodo `visualizzaProiezioneDettagliata()` della classe `Proiezione` e effettua la `return` in cui restituisce il vettore contenente le proiezioni risultato della ricerca. Nel caso in cui la ricerca non avesse risultati allora l'applicazione comunica all'utente che non è possibile visualizzare in dettaglio alcuna proiezione ed effettuerà `return null`.

Nel caso in cui l'utente desiderasse effettuare la ricerca di una proiezione per titolo film è sufficiente che inserisca il titolo (anche parziale) come scelta nel menù iniziale e il programma chiama il metodo `cercaProiezionePerTitolo()` della classe `Guest`, dandogli come parametro il titolo inserito dall'utente. Il metodo effettua la ricerca di una proiezione che abbia titolo compatibile con il titolo deciso dall'utente, accedendo al file `proiezioni.csv` per leggere le informazioni sulle proiezioni.

```
// Metodo cercaProiezionePerTitolo()
/**
 * Questo metodo permette la ricerca per titolo di proiezioni dal file proiezioni.csv ed eventualmente
 * la visualizzazione dettagliata di una di quelle trovate
 * Il metodo effettua stampo a video e se serve restituisce il vettore di oggetti Proiezione corrispondenti alle proiezioni trovate con la ricerca,
 * null nel caso in cui la ricerca avesse 0 risultati
 * Restituisce il titolo (anche parziale) usato per cercare una proiezione.
 * Restituisce Array di oggetti Proiezione contenente gli oggetti Proiezione corrispondenti alle proiezioni trovate con la ricerca per titolo,
 * null nel caso in cui la ricerca avesse 0 risultati
 * @throws FileNotFoundException Eccezione legata all'uso del file proiezioni.csv.
 */
public Proiezione[] cercaProiezionePerTitolo(String titoloCercato) throws FileNotFoundException {

    int limitTotale = 100; //Limita numero risultati della ricerca.
    int numRisRicerca = 0; //Contatore numero risultati.
    boolean proiezione; //Variabile boolean per dire se la proiezione che si sta considerando attualmente rispetta i criteri ricerca.

    Scanner kg = new Scanner(System.in);
    String inputStringa;
    boolean inputStringaOk;

    Scanner scFile = new Scanner(new File("../data/proiezioni.csv")); //scFile è lettore file proiezioni.
    scFile.useDelimiter("\n"); //Il separatore per distinguere una "cosa" letta dal file dalla successiva è l'a capo, quindi ogni .next() legge una riga del file.
    scFile.next(); //Salto la prima riga del file proiezioni che è l'intestazione.
    int numRigheIntestazione = 1;

    Proiezione[] risultatiCerca = new Proiezione[limitTotale]; //Vettore che rappresenta il risultato della ricerca cioè contiene le proiezioni che rispettano il...
    //...criterio scelto.

    String titoloLowerCercato, titoloLowerEsistente; //Variabili che conterranno i titoli cercati e estratti dalla proiezione corrente messi a minuscolo.
    String titoloUpper; //Non seguito la convenzione titoli.

    System.out.println("Ricerca proiezione con titolo film " + titoloCercato);
```

Figura 2.18: Metodo per la ricerca di una proiezione per titolo

Il metodo `cercaProiezionePerTitolo()` ha come parametro una stringa ossia il titolo da cercare e come valore di ritorno restituisce un array di oggetti di tipo `Proiezione` il quale conterrà gli oggetti `Proiezione` che rappresentano le proiezioni che sono risultato della ricerca, ossia il cui titolo corrisponde al parametro del metodo, oppure `null` nel caso in cui la ricerca non abbia risultati (perché nessuna proiezione rispetta i requisiti dell'utente). Il metodo inizia con le dichiarazioni di alcune variabili necessarie allo svolgimento del metodo. Dopodiché è presente il ciclo `while` `while(scFile.hasNext())` in cui è contenuta la ricerca (`while(scFile.hasNext())` va interpretato come “while(il file `proiezioni.csv` ha ancora qualcosa da leggere)” e quindi cioè “while(ci sono proiezioni da controllare)”).

```

System.out.println("Ricerca proiezione con titolo film : " + titoloCercato);
while(scFile.hasNext()) { //Ciclo per leggere una proiezione dal file delle proiezioni e verificare se rispetta requisiti.
    proiezioneOk = false;
    Proiezione proiezione = estraiProiezione(scFile, numvirgoleIntestaz); //Estraggo una proiezione dal file delle proiezioni e la metto in proiezione.
    //Inizio blocco per confronto tra criterio inserito e stesso criterio nella riga/proiezione letta da file.
    //Metto entrambi i titoli (quello ricercato e quello della proiezione considerata attualmente) a minuscolo per non far contare le maiuscole.
    titoloEmploowercase = proiezione.getTitolo().toLowerCase();
    titoloRiclowercase = titoloCercato.toLowerCase();
    //Confronto titoli.
    if(titoloRiclowercase.length() == titoloEmploowercase.length()) { //Caso titolo cercato ha lunghezza uguale al titolo proiezione corrente.
        if(titoloEmploowercase.compareTo(titoloRiclowercase) == 0)
            proiezioneOk = true;
    }
    if(titoloRiclowercase.length() < titoloEmploowercase.length()) { //Caso titolo cercato ha lunghezza minore del titolo proiezione corrente...
        //...quindi devo vedere se il titolo della proiezione corrente contiene il titolo cercato; per farlo uso titoloParz che...
        //...contiene man mano pezzi del titolo della proiezione corrente lunghi quanto il titolo cercato.
        //Se se cerco "evu", che è lungo 3, titoloParz conterrà man mano tutti i pezzi di lunghezza 3 del titolo della proiezione corrente per vedere se...
        //...uno di questi pezzi è "evu".
        for(int i=0; i<titoloEmploowercase.length(); i++) { //for per vedere se il titolo della proiezione corrente...
            titoloParz = titoloEmploowercase.substring(i, i+titoloRiclowercase.length());
            //contiene il titolo cercato.
            if(titoloParz.compareTo(titoloRiclowercase) == 0)
                proiezioneOk = true;
        }
    }
}
//Fine blocco confronto titoli.

```

Figura 2.19: Ciclo (while(scFile.hasNext()))

In particolare, nel ciclo while(scFile.hasNext()) il programma:

- ottiene una proiezione dal file proiezioni.csv tramite il metodo estraiProiezione() della classe Guest; il metodo estraiProiezione() restituisce un oggetto di tipo Proiezione che rappresenta la proiezione letta/estratta dal file proiezioni.csv
- verifica se la proiezione letta/estratta ha un titolo film compatibile con il titolo cercato
- nel caso in cui la proiezione rispetti i requisiti verifica se si è raggiunto il limite massimo di risultati della ricerca. In caso affermativo il programma segnala all'utente il raggiungimento del limite e quindi l'impossibilità di proseguire la ricerca, richiede di inserire dei criteri più restrittivi in una nuova ricerca e interrompe la ricerca corrente tramite la return con cui restituisce il vettore delle proiezioni trovate fino a quel momento. In caso contrario il programma salva la proiezione controllata in un vettore e visualizza la proiezione all'utente, numerandola e mostrandone le informazioni con il metodo visualizzaProiezione() della classe Proiezione.

Al termine della ricerca il programma informa l'utente del numero di risultati della ricerca. Il programma procede con l'if-else if(numRisRicerca>0)-else. L'if-else if(numRisRicerca>0)-else verifica se la ricerca ha avuto almeno un risultato o meno. Nel caso in cui la ricerca avesse risultati allora l'applicazione chiederà all'utente se desidera visualizzare in dettaglio una delle proiezioni trovate tramite la ricerca e, in caso affermativo, richiederà di inserire il numero della proiezione che desidera visualizzare in dettaglio. Inserito il numero di una delle proiezioni il programma permette all'utente di visualizzare nel dettaglio la proiezione scelta tramite il metodo visualizzaProiezioneDettagliata() della classe Proiezione ed effettua la return in cui restituisce il vettore contenente le proiezioni risultato della ricerca. Nel caso in cui la ricerca non avesse risultati allora l'applicazione comunica all'utente che non è possibile visualizzare in dettaglio alcuna proiezione ed effettuerà return null.

Logout

Quando si effettua il logout dall'applicazione, corrispondente all'input 0 da parte dell'utente nel menù principale, viene inizializzata la variabile boolean On a false ed eseguito un comando continue, in questo modo il codice salta alla successiva esecuzione del ciclo while (On=true) la cui condizione non sarà verificata e dunque il ciclo si arresterà, terminando l'applicazione; quando il ciclo di esecuzione principale termina viene mostrato a schermo un messaggio di arrivederci.

```

} //Fine while che legge il file delle proiezioni e controlla le proiezioni.
System.out.println("la ricerca ha dato " + numRisRicerca + " risultati");

if (numRisRicerca > 0) { //Inizio blocco che svolge funzionalità visualizzare in dettaglio una delle proiezioni cercate.
    String sceltaVisualizzaDettagli = "0"; //Variabile per scelta se visualizzare in dettaglio una delle proiezioni cercate, inizializ default a "0" cioè no.
    int sceltaNumProiezVisualiz = 1; //Variabile che conterrà il numero della proiezione scelta da visualizzare in dettaglio, inizializ default a 1 cioè...
    //...la prima, che sicuramente c'è se si sono entrati nel blocco if in cui c'è codice si trova allora c'è almeno 1 ris.
    System.out.println("Si desidera visualizzare i dettagli di una delle proiezioni cercate?");
    System.out.println("Inserire 1 se sì, 0 altrimenti");
    do { //Inizio ciclo do while per chiedere il numero della proiezione da visualizzare in dettaglio. Il while è while(true) (sk deve andare avanti finchè...
        //...l'utente inserisce una scelta valida).
        sceltaVisualizzaDettagli = sc.nextline();
        sceltaNumProiezVisualiz = sc.nextInt();

        switch(sceltaVisualizzaDettagli) {
            case "0": //Caso visualiz dettagliata di una delle proiezioni cercate rifiutata.
                return riskRicerca;

            case "1": //Caso visualiz dettagliata di una delle proiezioni cercate richiesta.
                System.out.println("Inserire il numero della proiezione di cui si desidera visualizzare i dettagli.");
                do {
                    inputStringIntOk=true;
                    try {
                        inputStringInt = sc.nextline();
                        sceltaNumProiezVisualiz = Integer.parseInt(inputStringInt);
                    } catch (NumberFormatException e) {
                        inputStringIntOk = false;
                        System.out.println("Non è stato inserito un numero intero. Inserire un numero intero. ");
                    }
                } while (inputStringIntOk == false);

                if (sceltaNumProiezVisualiz <= 0 || sceltaNumProiezVisualiz > numRisRicerca)
                    System.out.println("Il numero inserito non è valido. Inserire un numero valido di una delle proiezioni cercate.");
                else {
                    riskRicerca[sceltaNumProiezVisualiz-1].visualizzaProiezioneDettagliata(); //C'è il -1 perchè all'utente le proiezioni sono visualiz...
                    //...numerate da 1 (e quindi anche la sua scelta), mentre nel vettore sono numerate da 0.
                    return riskRicerca;
                }
            }
        } while (true);

        default:
            System.out.println("l'opzione scelta non è valida. Inserire un'opzione valida.");
    } while (true); //Fine ciclo per chiedere il numero della proiezione da visualizzare in dettaglio. Il while è while(true) sk deve andare avanti finchè...
    //...l'utente inserisce una scelta valida.
} else {
    System.out.println("la ricerca non ha risultati quindi non è possibile visualizzare i dettagli di una delle proiezioni cercate");
    return null;
} //Fine blocco che svolge funzionalità visualizzare in dettaglio una delle proiezioni cercate.

} //Fine metodo cercaProiezionePerTitolo().

```

Figura 2.20: If-else if(numRisRicerca>0)-else

```

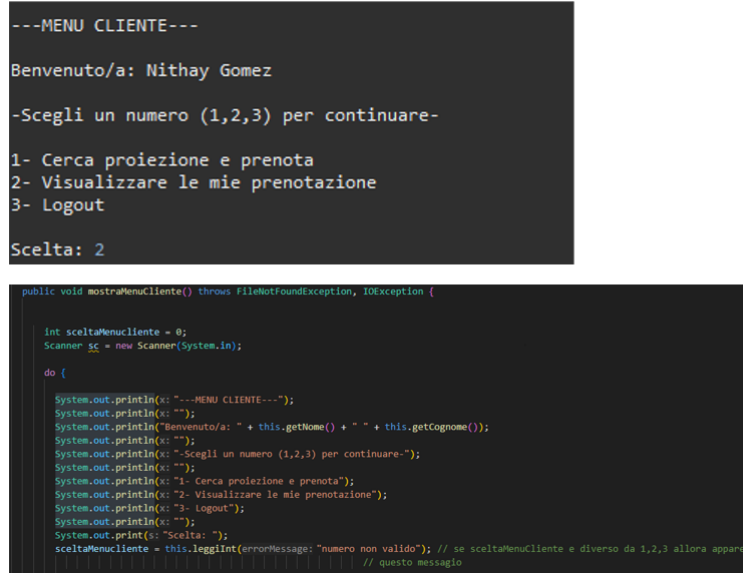
1160         break;
1161     }
1162     break; //fine accesso modalità guest
1163 case "0": //logout
1164     On=false; // spengo il ciclo di esecuzione while(On==true)
1165     continue; //fine logout

```

Figura 2.21: Logout

2.2.3 Funzionalità dell'utente di tipo cliente

Il menu del cliente è fatto da stampe `System.out.println` che mostra al cliente ogni opzione da scegliere



The image shows two parts: a terminal output of a menu and a snippet of Java code that generates it.

```

---MENU CLIENTE---
Benvenuto/a: Nithay Gomez
-Scegli un numero (1,2,3) per continuare-
1- Cerca proiezione e prenota
2- Visualizzare le mie prenotazione
3- Logout
Scelta: 2
  
```

```

public void mostraMenuCliente() throws FileNotFoundException, IOException {
    int sceltaMenuCliente = 0;
    Scanner sc = new Scanner(System.in);
    do {
        System.out.println("---MENU CLIENTE---");
        System.out.println("");
        System.out.println("Benvenuto/a: " + this.getNome() + " " + this.getCognome());
        System.out.println("");
        System.out.println("-Scegli un numero (1,2,3) per continuare-");
        System.out.println("");
        System.out.println("1- Cerca proiezione e prenota");
        System.out.println("2- Visualizzare le mie prenotazione");
        System.out.println("3- Logout");
        System.out.println("");
        System.out.print("Scelta: ");
        sceltaMenuCliente = this.leggiInt(errorMessage: "numero non valido"); // se sceltaMenuCliente è diverso da 1,2,3 allora appare // questo messaggio
    } while (sceltaMenuCliente != 1 & sceltaMenuCliente != 2 & sceltaMenuCliente != 3);
}
  
```

Figura 2.22: Menù cliente

Ricerca di una proiezione ed effettuare una prenotazione

Selezionando l'opzione 1, il sistema invoca il metodo `cercaProiezione()` della classe `Guest`, che restituisce l'elenco delle proiezioni disponibili sotto forma di array. L'utente seleziona quindi la proiezione desiderata tramite `Guest.selezionaProiezDaRicerca(risultatoRicerca)`; se la selezione non produce alcun risultato (valore null), l'operazione viene interrotta e il cliente torna al menu iniziale. In caso di selezione valida, il metodo `chiediNumeroDiPosti(proiezioneSelezionata)` chiede all'utente quanti posti desidera prenotare. Se il numero richiesto non è disponibile, la prenotazione viene annullata e l'utente torna al menu iniziale; altrimenti la prenotazione viene creata tramite `creaPrenotazione(proiezioneSelezionata, numeroPostiRichiesti)`. Il metodo `creaPrenotazione(Proiezione proiezioneSelezionata, int posti)` della classe `Cliente` verifica innanzitutto la disponibilità dei posti richiesti tramite `proiezioneSelezionata.haPostiDisponibili(posti)`. In caso affermativo, calcola e mostra il prezzo totale dei biglietti e chiede conferma all'utente (S/N). Se la risposta inizia con il carattere 'S', viene istanziato un nuovo oggetto `Prenotazione` con i dati del cliente e della proiezione, e lo si salva su file tramite `Prenotazione.aggiungiPrenotazioneAlCSV(prenotazione)`; in caso contrario, o se i posti non sono disponibili, la prenotazione viene annullata.

Il metodo statico `aggiungiPrenotazioneAlCSV(Prenotazione prenotazione)` apre il file in modalità append tramite un `FileWriter`, in modo da non sovrascrivere le righe già presenti, e compone una nuova riga concatenando i campi della prenotazione separati da virgola, secondo il formato del file CSV. La data della proiezione viene formattata tramite `DateTimeFormatter.ofPattern("yyyy-MM-dd'T'HH:mm:ss")`. La riga viene quindi scritta su file; eventuali errori di lettura o scrittura vengono intercettati con un blocco catch (`IOException` e), che ne stampa i dettagli e restituisce false.

```

public boolean creaPrenotazione(Proiezione proiezioneSelezionata, int posti) {
    Scanner sc = new Scanner(System.in);

    boolean haPostiDisponibili = proiezioneSelezionata.haPostiDisponibili(posti); // se crea un boolean per sapere se ci
    // sono posti disponibili

    if (haPostiDisponibili == true) { // se la variabile e true se procede a fare la prenotazione
        String sceltaPrenota;
        System.out
            .println("Il prezzo totale delle tue prenotazione è: " + proiezioneSelezionata.getPrezzoBiglietto() * posti);
        System.out.println(x: "");
        System.out.println(x: "sei sicuro/a che vuoi prenotare? (S/N): ");
        sceltaPrenota = sc.nextLine();

        if (sceltaPrenota.toUpperCase().charAt(index: 0) == 'S') { // se verifica che il carattere sia S, in caso in cui siamo
            // uguale se procede a fare la nuova prenotazione

            Prenotazione prenotazione = new Prenotazione(this.getIdUtente(), this.getNome(), this.getCognome(),
                proiezioneSelezionata.getDataOra(),
                proiezioneSelezionata.getFilm().getTitolo(), posti, proiezioneSelezionata.getPrezzoBiglietto());

            return Prenotazione.aggiungiPrenotazioneAlCSV(prenotazione);
        } else {
            System.out.println(x: "prenotazione annullata");
            return false;
        }
    } else {
        return false;
    }
}

```

Figura 2.23: Il metodo creaPrenotazione(Proiezione proiezioneSelezionata, int posti)

```

public boolean creaPrenotazione(Proiezione proiezioneSelezionata, int posti) {
    Scanner sc = new Scanner(System.in);

    boolean haPostiDisponibili = proiezioneSelezionata.haPostiDisponibili(posti); // se crea un boolean per sapere se ci
    // sono posti disponibili

    if (haPostiDisponibili == true) { // se la variabile e true se procede a fare la prenotazione
        String sceltaPrenota;
        System.out
            .println("Il prezzo totale delle tue prenotazione è: " + proiezioneSelezionata.getPrezzoBiglietto() * posti);
        System.out.println(x: "");
        System.out.println(x: "sei sicuro/a che vuoi prenotare? (S/N): ");
        sceltaPrenota = sc.nextLine();

        if (sceltaPrenota.toUpperCase().charAt(index: 0) == 'S') { // se verifica che il carattere sia S, in caso in cui siamo
            // uguale se procede a fare la nuova prenotazione

            Prenotazione prenotazione = new Prenotazione(this.getIdUtente(), this.getNome(), this.getCognome(),
                proiezioneSelezionata.getDataOra(),
                proiezioneSelezionata.getFilm().getTitolo(), posti, proiezioneSelezionata.getPrezzoBiglietto());

            return Prenotazione.aggiungiPrenotazioneAlCSV(prenotazione);
        } else {
            System.out.println(x: "prenotazione annullata");
            return false;
        }
    } else {
        return false;
    }
}

```

Figura 2.24: aggiungiPrenotazioneAlCSV(Prenotazione prenotazione)

Modificare una prenotazione

Selezionando l'opzione 2 dal menu principale, il sistema entra in un sotto-menu dedicato alla gestione delle prenotazioni del cliente, anch'esso realizzato con un ciclo do-while e uno switch a quattro rami mostrato in figura 2.25

```
case 2:
    int sceltaPrenotazioni;
    do {
        /*
         * interfaccia di prenotazione
         */
        System.out.println(x: "---LE MIE PRENOTAZIONI---");
        System.out.println(x: "");
        System.out.println(x: "-Scegli un numero per continuare-");
        System.out.println(x: "");
        System.out.println(x: "1- Visualizzare prenotazioni");
        System.out.println(x: "2- Modificare prenotazione");
        System.out.println(x: "3- Eliminare prenotazione");
        System.out.println(x: "0- INDIETRO");
        System.out.println(x: "");
        System.out.println(x: "Scelta");
        sceltaPrenotazioni = this.leggiInt(errorMessage: "numero non valido");
```

Figura 2.25: Lo switch case del menù di modifica prenotazioni

Il ramo case 1 richiama il metodo `visualizzaLeMiePrenotazioni()`, che elenca a video le prenotazioni del cliente. Il sotto-menu resta attivo finché l'utente non inserisce 0 per tornare indietro. Il ramo case 2 mostra il dettaglio della prenotazione selezionata e propone due possibilità di modifica: il numero di posti oppure la data della proiezione. La scelta viene letta anch'essa tramite `leggiInt`. Se l'utente sceglie di modificare il numero di posti (case 1), viene invocato il metodo `chiediQuantiPostiPrenotareEmodifica(proiezioneSelezionata, prenotazioneSelezionata)`, che gestisce l'aggiornamento del numero di posti richiesti per la proiezione. Se invece l'utente sceglie di modificare la data (case 2), il sistema recupera con `cercaProiezPerCambioPrenotaz(prenotazioneSelezionata.getProiezione_Titolo())` tutte le proiezioni disponibili per lo stesso film, le presenta in un elenco numerato tramite `Guest.selezionaProiezDaRicerca(listaProiezioni)` e infine richiama nuovamente `chiediQuantiPostiPrenotareEmodifica(...)` per completare l'aggiornamento della prenotazione.

Cancellare una prenotazione

Il ramo case 3 carica l'intero archivio delle prenotazioni tramite `Prenotazione.caricaPrenotazioni()` e ne filtra il contenuto con `Prenotazione.TrovaPrenotazioniConNomeECognome(nome, cognome, prenotazioniC, tutte)`, isolando le sole prenotazioni intestate al cliente collegato. Se non viene trovata alcuna prenotazione futura, l'operazione viene interrotta e l'utente torna al menu precedente; in caso contrario, il sistema propone l'elenco delle prenotazioni tra cui scegliere quella da eliminare.

```
case 3:
    int sceltaElimina;
    System.out.println(x: "---ELIMINA PRENOTAZIONE---");
    System.out.println(x: "");
    ArrayList<Prenotazione> prenotazioniC = Prenotazione.caricaPrenotazioni(); // legge il file CSV e
    // carica la lista
    // completa di tutte le
    // prenotazioni di tutti
    // gli utenti
    ArrayList<Prenotazione> prenotazioniClienti = new ArrayList<Prenotazione>();
    prenotazioniClienti = Prenotazione.TrovaPrenotazioniConNomeECognome(this.getNome(), this.getCognome(),
    prenotazioniC, tutte: false); // filtra la lista generale cercando soltanto le prenotazioni
    // che corrispondono al nome e cognome
    System.out.println(x: "");
    System.out.println(x: "Que prenotazione vuoi eliminare? ");
    System.out.println(x: "");
    int indice = 1;
    if (prenotazioniClienti.size() == 0) {
        System.out
        .println(x: "non ci sono prenotazioni disponibili di questa data in poi (tornando indietro)");
        break;
```

Figura 2.26: Lo switch case del menù di cancellazione di una prenotazione

Il metodo statico `eliminaPrenotazioneDalCSV(int idPrenotazione)` legge il file riga per riga tramite `BufferedReader`, conservando l'intestazione e tutte le righe non vuote in una lista temporanea. Per ciascuna riga, l'identificativo viene confrontato con `idPrenotazione`: in caso di corrispondenza la riga viene esclusa (tramite `continue`), mentre tutte le altre righe vengono mantenute e successivamente riscritte sul file, ottenendo così l'eliminazione della prenotazione richiesta.

2.2.4 Funzionalità dell'utente di tipo proiezionista

Nel caso in cui nel menù iniziale l'utente scelga di effettuare il log in accedendo con credenziali da proiezionista verrà visualizzato il menù mostrato in figura 2.27

```
Benvenuto**PR01
Cosa vuoi fare oggi?

Digita 1 per aggiungere una proiezione al palinsesto
Digita 2 per rimuovere una proiezione dal palinsesto
Digita 3 per modificare la data di una proiezione dal palinsesto
Digita 0 per effettuare il logout
```

Figura 2.27: Menù bigliettaio

Ricerca di una proiezione

L'inserimento di una proiezione nel palinsesto avviene tramite il metodo `aggiungiProiezioneAlPalinsesto(Proiezione, ArrayList<Proiezione>)` della classe `proiezionista` mostrato in figura 2.28, che riceve in ingresso il nuovo oggetto `Proiezione` e l'`ArrayList<Proiezione>` che rappresenta il palinsesto.

Il metodo

- calcola l'intervallo orario dello spettacolo compreso tra `inizioNuova (dataOra)` e `fineNuova (dataOra + durata film)`.
- esegue un ciclo di verifica su tutte le proiezioni esistenti per rilevare intersezioni temporali tramite la condizione: `inizioNuova.isBefore(fineEsistente) && fineNuova.isAfter(inizioEsistente)`.

In assenza di conflitti, l'elemento viene aggiunto all'`ArrayList`, riordinato tramite `Collections.sort()` e salvato su file CSV tramite `Proiezione.salvaProiezioni()`.

Modificare una proiezione

La modifica della data e/o dell'orario di una proiezione si esegue con il metodo `modificaDataOraProiezione(String, LocalDateTime, LocalDateTime ArrayList<Proiezione>)` della classe `proiezionista` mostrato in figura 2.29, che riceve in ingresso il titolo del film passato come `String`, la data e l'ora correnti e nuove in formato `LocalDateTime` e l'`ArrayList<Proiezione>` che rappresenta il palinsesto.

Il metodo individua l'oggetto target all'interno del palinsesto e valuta la validità della nuova data/ora proposta:

```

public boolean aggiungiProiezioneAlPalinsesto(Proiezione nuovaProiezione, ArrayList<Proiezione> palinsesto) {

    if (nuovaProiezione == null || palinsesto == null) {
        System.out.println("ERRORE: Proiezione o palinsesto non validi.");
        return false;
    }

    // Controllo 1: l'anno del film non deve essere nel futuro
    int annoCorrente = Year.now().getValue();
    if (nuovaProiezione.getFile().getAnno() > annoCorrente) {
        System.out.println("ERRORE: Impossibile aggiungere un film con anno di pubblicazione nel futuro ("
            + nuovaProiezione.getFile().getAnno() + ")!");
        return false;
    }

    // Controllo 2: La data della proiezione deve essere futura
    if (nuovaProiezione.getDataOra().isBefore(LocalDateTime.now())) {
        System.out.println("ERRORE: Impossibile inserire una proiezione nel passato! Data inserita: " + nuovaProiezione.getDataOra());
        return false;
    }

    // Controllo 3: Verifica se l'orario si sovrappone a qualche altro film in sala
    for (Proiezione p : palinsesto) {
        LocalDateTime inizioEsistente = p.getDataOra();
        LocalDateTime fineEsistente = inizioEsistente.plusMinutes(p.getFile().getDurata());

        LocalDateTime inizioNuova = nuovaProiezione.getDataOra();
        LocalDateTime fineNuova = inizioNuova.plusMinutes(nuovaProiezione.getFile().getDurata());

        // Controlla se le due proiezioni si incrociano
        if (inizioNuova.isBefore(fineEsistente) && fineNuova.isAfter(inizioEsistente)) {
            System.out.println("ERRORE: Il cinema e' gia' occupato in quell'orario dal film: " + p.getFile().getTitolo());
            return false;
        }
    }

    // Aggiunge la proiezione, riordina la lista e salva sul file CSV
    palinsesto.add(nuovaProiezione);
    Collections.sort(palinsesto);

    Proiezione.salvaProiezioni(palinsesto);

    System.out.println("Proiezione di \"" + nuovaProiezione.getFile().getTitolo() + "\" inserita con successo.");
    return true;
}

```

Figura 2.28: Il metodo `aggiungiProiezioneAlPalinsesto(Proiezione, ArrayList<Proiezione>)`

- ottiene una prenotazione dal file `Prenotazioni.csv` tramite il metodo `estraiPrenotazione()` della classe `Bigliettaio`; il metodo `estraiPrenotazione()` restituisce un oggetto di tipo `Prenotazione` che rappresenta la prenotazione letta/estratta dal file `Prenotazioni.csv`
- verifica che la nuova data sia successiva a `LocalDateTime.now()`
- riesegue il controllo di sovrapposizione oraria ignorando nel ciclo il confronto del film con se stesso (`if (p == proiezioneDaModificare) continue;`)

Se le verifiche hanno esito positivo, il campo `dataOra` viene aggiornato tramite `setter`, la lista viene riordinata cronologicamente e salvata su file.

Cancellare una proiezione

La rimozione di una proiezione nel `palinsesto` avviene invocando il metodo `rimuoviProiezione-DalPalinsesto(String, LocalDateTime, ArrayList<Proiezione>, ArrayList<Prenotazione>)` della classe `proiezionista` mostrato in figura 2.30, che riceve in ingresso il titolo del film passato come `String`, la data e l'ora in formato `LocalDateTime`, l'`ArrayList<Proiezione>` che rappresenta il `palinsesto` e l'`ArrayList<Prenotazione>` che rappresenta l'elenco di tutte le prenotazioni.

Il metodo scansiona la lista delle prenotazioni attive per verificare se esistono record corrispondenti allo stesso titolo e alla stessa data/ora: se la ricerca dà esito positivo (trovata almeno una prenotazione), il metodo interrompe l'esecuzione restituendo `false`, in caso contrario l'oggetto viene rimosso dal `palinsesto` e la modifica viene persistita su file.


```

public boolean modificaDataOraProiezione(String titoloFilm, LocalDateTime dataOraAttuale, LocalDateTime nuovaDataOra, ArrayList<Proiezione> palinsesti) {
    if (titoloFilm == null || dataOraAttuale == null || nuovaDataOra == null || palinsesti == null) {
        System.out.println("ERRORE: Parametri non validi per la modifica.");
        return false;
    }

    // Controllo che la nuova data non sia nel passato
    if (nuovaDataOra.isBefore(LocalDateTime.now())) {
        System.out.println("ERRORE: Impossibile spostare una proiezione nel passato! Nuova data inserita: " + nuovaDataOra);
        return false;
    }

    // Cerca la proiezione giusta nella lista
    Proiezione proiezioneDaModificare = null;
    for (Proiezione p : palinsesti) {
        if (p.getFilm().getTitle().equalsIgnoreCase(titoloFilm) && p.getDataOra().equals(dataOraAttuale)) {
            proiezioneDaModificare = p;
            break;
        }
    }

    if (proiezioneDaModificare == null) {
        System.out.println("ERRORE: Nessuna proiezione trovata per il film '" + titoloFilm + "' in data: " + dataOraAttuale);
        return false;
    }

    // Controllo se il nuovo orario si sovrappone con un altro film
    int durataNuova = proiezioneDaModificare.getFilm().getDurata();
    LocalDateTime fineNuova = nuovaDataOra.plusMinutes(durataNuova);

    for (Proiezione p : palinsesti) {
        // Salta il controllo con se stessa
        if (p == proiezioneDaModificare) {
            continue;
        }

        LocalDateTime inizioEsistente = p.getDataOra();
        LocalDateTime fineEsistente = inizioEsistente.plusMinutes(p.getFilm().getDurata());

        if (nuovaDataOra.isBefore(fineEsistente) && fineNuova.isAfter(inizioEsistente)) {
            System.out.println("ERRORE: Impossibile spostare la proiezione. Il cinema e' gia' occupato dal film: " + p.getFilm().getTitle());
            return false;
        }
    }

    // Aggiorna l'orario, riordina e salva sul CSV
    proiezioneDaModificare.setDataOra(nuovaDataOra);
    Collections.sort(palinsesti);

    Proiezione.salvaProiezioni(palinsesti);

    System.out.println("Data e ora della proiezione di '" + titoloFilm + "' aggiornate con successo a: " + nuovaDataOra);
    return true;
}

```

Figura 2.29: Il metodo `modificaDataOraProiezione(String, LocalDateTime, LocalDateTime ArrayList<Proiezione>)`

```

public boolean rimuoviProiezioneDalPalinsesto(String titoloFilm, LocalDateTime dataOraProiezione, ArrayList<Proiezione> palinsesti, ArrayList<Prenotazione> listaPrenotazioni) {
    if (titoloFilm == null || dataOraProiezione == null || palinsesti == null || listaPrenotazioni == null) {
        System.out.println("ERRORE: Parametri non validi per la rimozione.");
        return false;
    }

    for (int i = 0; i < palinsesti.size(); i++) {
        Proiezione p = palinsesti.get(i);

        // Confronta sia il titolo che la data/ora esatta
        if (p.getFilm().getTitle().equalsIgnoreCase(titoloFilm) && p.getDataOra().equals(dataOraProiezione)) {
            // Cerca se ci sono prenotazioni per questa proiezione
            for (Prenotazione pr : listaPrenotazioni) {
                if (pr.getProiezione_titolo().equalsIgnoreCase(p.getFilm().getTitle()) &&
                    pr.getProiezione_Data().equals(p.getDataOra())) {
                    System.out.println("ERRORE: Impossibile rimuovere la proiezione di '" + titoloFilm + "' del '" + dataOraProiezione + "'. Ci sono prenotazioni attive!");
                    return false;
                }
            }

            // Rimuove la proiezione e aggiorna il file
            palinsesti.remove(i);
            Proiezione.salvaProiezioni(palinsesti);

            System.out.println("La proiezione di '" + titoloFilm + "' del '" + dataOraProiezione + "' e' stata rimossa con successo.");
            return true;
        }
    }

    System.out.println("Nessuna proiezione trovata per il film '" + titoloFilm + "' nella data/ora: " + dataOraProiezione);
    return false;
}

```

Figura 2.30: Il metodo `rimuoviProiezioneDalPalinsesto(String, LocalDateTime, ArrayList<Proiezione>, ArrayList<Prenotazione>)`

2.2.5 Funzionalità dell'utente di tipo bigliettaio

Nel caso in cui nel menù iniziale l'utente scelga di effettuare il log in e acceda con credenziali da bigliettaio verrà visualizzato il menù mostrato in figura 2.31

```
Benvenuto$$BIG1
Cosa vuoi fare oggi?

Digita 1 per visualizzare le prenotazioni nella data odierna

Digita 2 per cercare una prenotazione

Digita 0 per effettuare il logout
```

Figura 2.31: Menù bigliettaio

Visualizzare tutte le prenotazioni in data odierna

Quando l'utente accede alla funzionalità di visualizzazione prenotazioni odierne il programma chiama il metodo `visualizzaPrenotazioniOdierna()` della classe `Bigliettaio`. Il metodo effettua la ricerca delle prenotazioni odierne, accedendo al file `Prenotazioni.csv` per leggere le informazioni sulle prenotazioni, e, se presenti, queste verranno visualizzate.

Il metodo non ha parametri e non ha valore di ritorno. Il metodo inizia con le dichiarazioni di alcune variabili necessarie allo svolgimento del metodo. Dopodichè è presente il ciclo `while(scFile.hasNext())` in cui è contenuta la ricerca e la stampa delle prenotazioni odierne (`while(scFile.hasNext())` va interpretato come “while(il file `Prenotazioni.csv` ha ancora qualcosa da leggere)” e quindi cioè “while(ci sono prenotazioni da controllare)”). In particolare, nel ciclo `while(scFile.hasNext())` il programma:

- ottiene una prenotazione dal file `Prenotazioni.csv` tramite il metodo `estraiPrenotazione()` della classe `Bigliettaio`; il metodo `estraiPrenotazione()` restituisce un oggetto di tipo `Prenotazione` che rappresenta la prenotazione letta/estratta dal file `Prenotazioni.csv`
- verifica se la prenotazione letta/estratta è odierna
- in caso affermativo visualizza la prenotazione all'utente, numerandola e mostrandone le informazioni con il metodo `toString()` della classe `Prenotazione` e mostrando il costo totale (ottenuto a partire dai campi `prezzo biglietto` e `numero posti` dell'oggetto `Prenotazione`)

Al termine della ricerca il programma informa l'utente del numero di risultati della ricerca.

Visualizzare le prenotazioni effettuate da un cliente

Quando l'utente accede alla funzionalità di ricerca di una prenotazione il programma chiama il metodo `cercaPrenotazione()` della classe `Bigliettaio`.

Il metodo effettua la ricerca di una prenotazione, secondo criteri decisi dall'utente, accedendo al file `Prenotazioni.csv` per leggere le informazioni sulle prenotazioni. Il metodo non ha parametri e come valore di ritorno restituisce un array di oggetti di tipo `Prenotazione` che conterrà gli oggetti `Prenotazione` che rappresentano le prenotazioni che sono risultato della ricerca, oppure

```

//Inizio metodo visualizzaPrenotazioniOdierne().
/**
 * Questo metodo permette di visualizzare le prenotazioni odierne accedendo al file Prenotazioni.csv.
 * @throws FileNotFoundException Eccezione legata all'uso del file Prenotazioni.csv.
 */
public void visualizzaPrenotazioniOdierne() throws FileNotFoundException {
//Questo metodo permette la visualizzazione delle prenotazioni odierne.

    int numRisRicerca = 0; //Contatore numero risultati.

    double costoTot=0;

    Scanner scFile = new Scanner(new File("../data/Prenotazioni.csv")); //scFile è lettore file prenotazioni.
    scFile.useDelimiter("\n"); //Il separatore per distinguere una "cosa" letta dal file dalla successiva è l'a-capo, quindi ogni .next le
    scFile.next(); //Salto la prima riga del file prenotazioni che è l'intestazione.
    int numvirgoleintestaz = 7; //Numero virgole intestazione file prenotazioni Prenotazioni.csv.

    LocalDate dataOdierna = LocalDateTime.now().toLocalDate();

    System.out.println("Visualizzazione prenotazioni odierne");

    while(scFile.hasNext()) { //Ciclo per leggere una prenotazione dal file delle prenotazioni, verificare se rispetta requisiti ed eventua

        Prenotazione prenotaz = estraiPrenotazione(scFile,numvirgoleintestaz); //Estraggo una prenotazione dal file delle prenotazioni e l

        //Confronto date e stampa.
        if (prenotaz.getProiezione_Data().toLocalDate().isEqual(dataOdierna) == true) {
            numRisRicerca++;
            System.out.println(numRisRicerca);
            System.out.println(prenotaz.toString(true));
            System.out.println("Prezzo biglietto:" + prenotaz.getPrezzoBiglietto() );
            costoTot = prenotaz.getPrezzoBiglietto() * prenotaz.getNPosti();
            System.out.println("Costo totale:" + costoTot);
        }
        //Fine blocco confronto date e stampa.

    } //Fine while che legge il file delle prenotazioni, controlla le prenotazioni ed eventualmente stampa.
    System.out.println("La ricerca ha dato " + numRisRicerca + " risultati");
}
//Fine metodo visualizzaPrenotazioniOdierne().

```

Figura 2.32: Metodo visualizzaPrenotazioniOdierne()

null nel caso in cui la ricerca non abbia risultati (perché nessuna prenotazione rispetta i requisiti dell'utente oppure perché la ricerca è stata annullata).

Il funzionamento di questo metodo è sostanzialmente analogo a quello di `cercaProiezione()` discusso nella sezione 2.2.2.

2.3 Data set di test

Capitolo 3

Limiti della soluzione proposta

Capitolo 4

Bibliografia